

FA7_Part 1: Exponential Distribution of Turnstile Failures in the FC Building

Lira Dabu, Ben Libres, Aldwin Montilla

2026-10-08

Contents

Importing and Organizing the Data	1
1. Verify if Exponential Distribution is Applicable	3
Are the events occurring randomly and independently over time?	3
Identify the average rate (λ) of event occurrence per unit of time	5
2. Compute Key Parameters	5
Mean (expected value) = $1/\lambda$	5
Probability Density Function (PDF)	6
Cumulative Distribution Function (CDF)	6
Probabilities (computed in R)	6
3. Visualize and Check the Fit	6
4. Report Findings	9
5. Interpret the Results	10
Determine the likelihood of turnstile failure in the FC Building	10
Implications	10

Importing and Organizing the Data

```
# 1. Import the data collected
raw <- read.csv("raw.csv", header = TRUE, check.names = FALSE)

# 2. Keep the 2 columns and clean the header names
turnstile <- raw[, 1:2]
names(turnstile) <- c("Observation", "Interarrival_sec")
```

```

# Convert to numeric safely: strip anything that is not a digit, dot or minus
to_num <- function(v) suppressWarnings(as.numeric(gsub("[^0-9.-]", "", as.character(v))))

turnstile$Observation      <- to_num(turnstile$Observation)
turnstile$Interarrival_sec <- to_num(turnstile$Interarrival_sec)

# Keep only real observation rows (drops blank/summary rows)
turnstile <- turnstile[!is.na(turnstile$Observation) &
                      !is.na(turnstile$Interarrival_sec), ]
rownames(turnstile) <- NULL

stopifnot("No valid inter-arrival values found - check the CSV columns" =
  nrow(turnstile) > 0)

# 3. Compute cumulative time from the inter-arrival times
turnstile$Cumulative_sec <- round(cumsum(turnstile$Interarrival_sec), 2)

# 4. Convert cumulative seconds to hh:mm:ss
secs <- round(turnstile$Cumulative_sec)
turnstile$Timestamp <- sprintf("%02d:%02d:%02d",
                               secs %/% 3600,
                               (secs %/% 3600) %/% 60,
                               secs %% 60)

# 5. Final organized table
turnstile <- turnstile[, c("Observation", "Interarrival_sec",
                          "Cumulative_sec", "Timestamp")]
print(turnstile, row.names = FALSE)

```

```

## Observation Interarrival_sec Cumulative_sec Timestamp
##          1           6.02           6.02 00:00:06
##          2          39.37          45.39 00:00:45
##          3          10.68          56.07 00:00:56
##          4          19.19          75.26 00:01:15
##          5           6.52          81.78 00:01:22
##          6           1.63          83.41 00:01:23
##          7          53.39         136.80 00:02:17
##          8           8.75         145.55 00:02:26
##          9           4.62         150.17 00:02:30
##         10          12.33         162.50 00:02:42
##         11          22.35         184.85 00:03:05
##         12          55.41         240.26 00:04:00
##         13           4.21         244.47 00:04:04
##         14          58.24         302.71 00:05:03
##         15           5.39         308.10 00:05:08
##         16           1.26         309.36 00:05:09
##         17         113.32         422.68 00:07:03
##         18          12.44         435.12 00:07:15
##         19          14.84         449.96 00:07:30
##         20           8.90         458.86 00:07:39
##         21           9.15         468.01 00:07:48
##         22          29.72         497.73 00:08:18
##         23         175.75         673.48 00:11:13

```

##	24	124.76	798.24	00:13:18
##	25	68.26	866.50	00:14:26
##	26	5.52	872.02	00:14:32
##	27	8.73	880.75	00:14:41
##	28	8.42	889.17	00:14:49
##	29	35.59	924.76	00:15:25
##	30	188.82	1113.58	00:18:34
##	31	12.36	1125.94	00:18:46
##	32	14.66	1140.60	00:19:01
##	33	16.99	1157.59	00:19:18
##	34	12.36	1169.95	00:19:30
##	35	58.91	1228.86	00:20:29
##	36	177.13	1405.99	00:23:26
##	37	114.73	1520.72	00:25:21
##	38	22.09	1542.81	00:25:43
##	39	61.89	1604.70	00:26:45

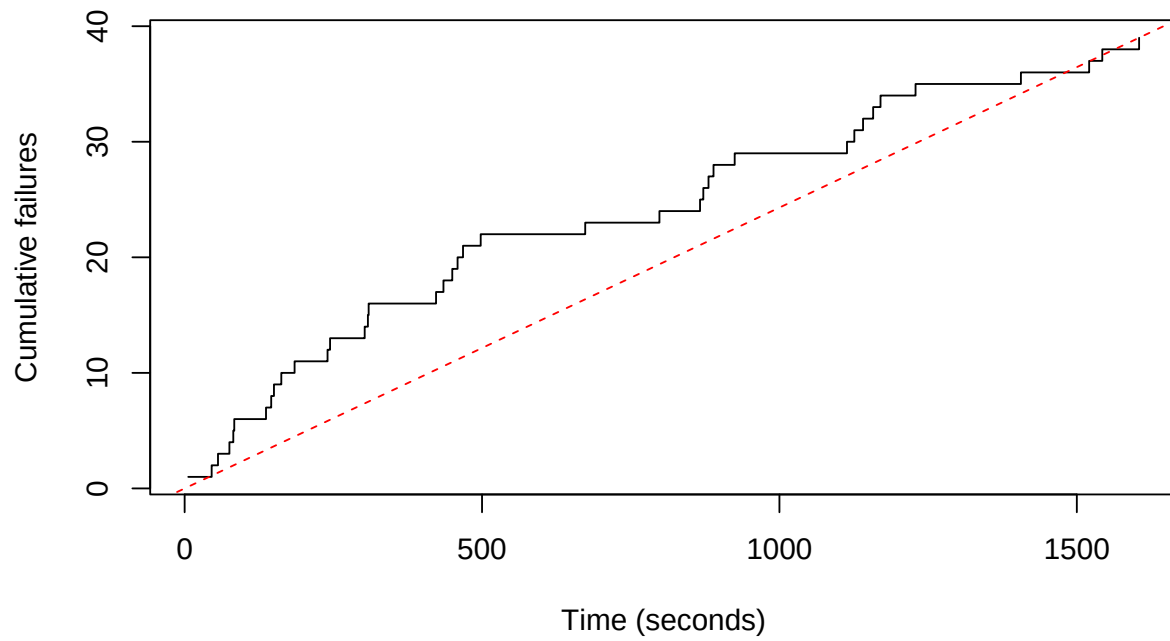
```
# Variable used throughout: waiting time (seconds) between consecutive failures
x      <- turnstile$Interarrival_sec
n      <- length(x)
lambda <- 1 / mean(x)           # estimated rate (failures per second)
EX     <- 1 / lambda            # mean waiting time (seconds)
```

1. Verify if Exponential Distribution is Applicable

Are the events occurring randomly and independently over time?

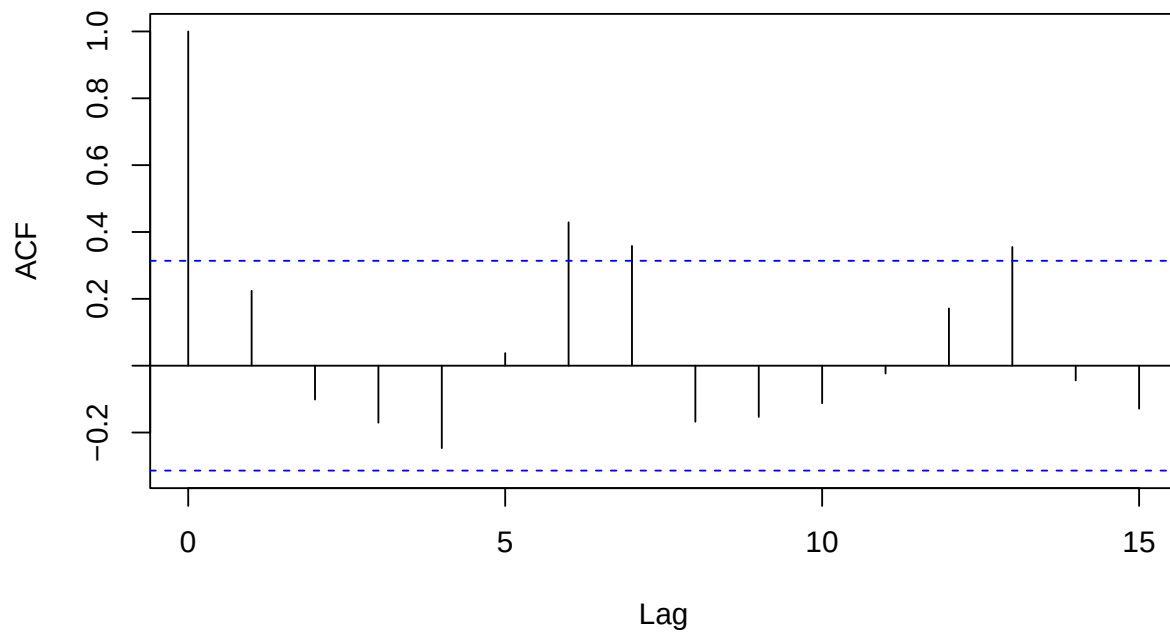
```
# RANDOM + steady rate: cumulative failures over time (red line = perfectly steady)
plot(turnstile$Cumulative_sec, turnstile$Observation, type = "s",
     xlab = "Time (seconds)", ylab = "Cumulative failures",
     main = "Cumulative failures over time")
abline(0, lambda, col = "red", lty = 2)
```

Cumulative failures over time



```
# INDEPENDENT: do gaps predict the next gap?  
acf(x, main = "ACF of inter-arrival times")
```

ACF of inter-arrival times



```
lag1 <- cor(x[-n], x[-1]) # lag-1 correlation
limit <- 2 / sqrt(n)      # approximate 95% limit
cat("Lag-1 correlation:", round(lag1, 3), "\n",
    "+/- limit (2/sqrt(n)):", round(limit, 3), "\n")
```

```
## Lag-1 correlation: 0.226
## +/- limit (2/sqrt(n)): 0.32
```

The occurrences of turnstile failures (unresponsive I.D. scanner or face recognition failure) occurred randomly and independently over time.

The autocorrelation plot shows that consecutive waiting times are not related (lag-1 correlation = 0.226, within the ± 0.32 limit), meaning that a prior error does not affect or result in a higher or lower probability of the next error event happening.

The cumulative failures plot shows that failures occurred throughout the observation period, but the pace was not perfectly steady. There were more failures early on than later because the observation was conducted during peak hours of student entries at around 12:00 noon, meaning it had a faster start of the event occurrences.

Since the events are random and independent, with only a mild variation in rate, the exponential distribution is a reasonable model for the waiting time between failures.

Identify the average rate (λ) of event occurrence per unit of time

```
# For an exponential distribution, mean waiting time = 1/lambda,
# so we estimate lambda as 1 divided by the sample mean.
lambda_sec <- 1 / mean(x) # failures per second
lambda_min <- lambda_sec * 60 # failures per minute

cat("Sample size (n):", n, "\n",
    "Average waiting time:", round(mean(x), 2), "seconds\n",
    "Rate (lambda):", round(lambda_sec, 5), "failures per second\n",
    "Rate (lambda):", round(lambda_min, 3), "failures per minute\n")
```

```
## Sample size (n): 39
## Average waiting time: 41.15 seconds
## Rate (lambda): 0.0243 failures per second
## Rate (lambda): 1.458 failures per minute
```

The average rate of failure is $\lambda = 0.0243$ failures per second (about 1.46 per minute), or one failure about every 41 seconds.

2. Compute Key Parameters

Mean (expected value) = $1/\lambda$

```
cat("lambda    =", round(lambda, 4), "failures per second\n",
    "Mean E[X] =", round(EX, 4), "seconds\n")
```

```
## lambda    = 0.0243 failures per second
## Mean E[X] = 41.1462 seconds
```

Probability Density Function (PDF)

$$f(t) = \lambda e^{-\lambda t}, \quad t \geq 0$$

```
# Height of the curve at a chosen time (curve is highest at 0 and decreases).
t0 <- 30                                # time to evaluate (seconds)
f <- lambda * exp(-lambda * t0)

cat("f(", t0, ") =", round(f, 4), "\n")
```

```
## f( 30 ) = 0.0117
```

Cumulative Distribution Function (CDF)

$$F(t) = P(X \leq t) = 1 - e^{-\lambda t}$$

```
# Probability that the next failure happens within t0 seconds.
F_cdf <- 1 - exp(-lambda * t0)

cat("P(X <=", t0, ") =", round(F_cdf, 4), "\n")
```

```
## P(X <= 30 ) = 0.5177
```

Probabilities (computed in R)

```
times <- c(10, 30, 60, 120)

prob_table <- data.frame(
  Time_sec      = times,
  P_within_t    = round(1 - exp(-lambda * times), 4),  # P(X <= t)
  P_longer_than_t = round(exp(-lambda * times), 4)      # P(X > t)
)
prob_table
```

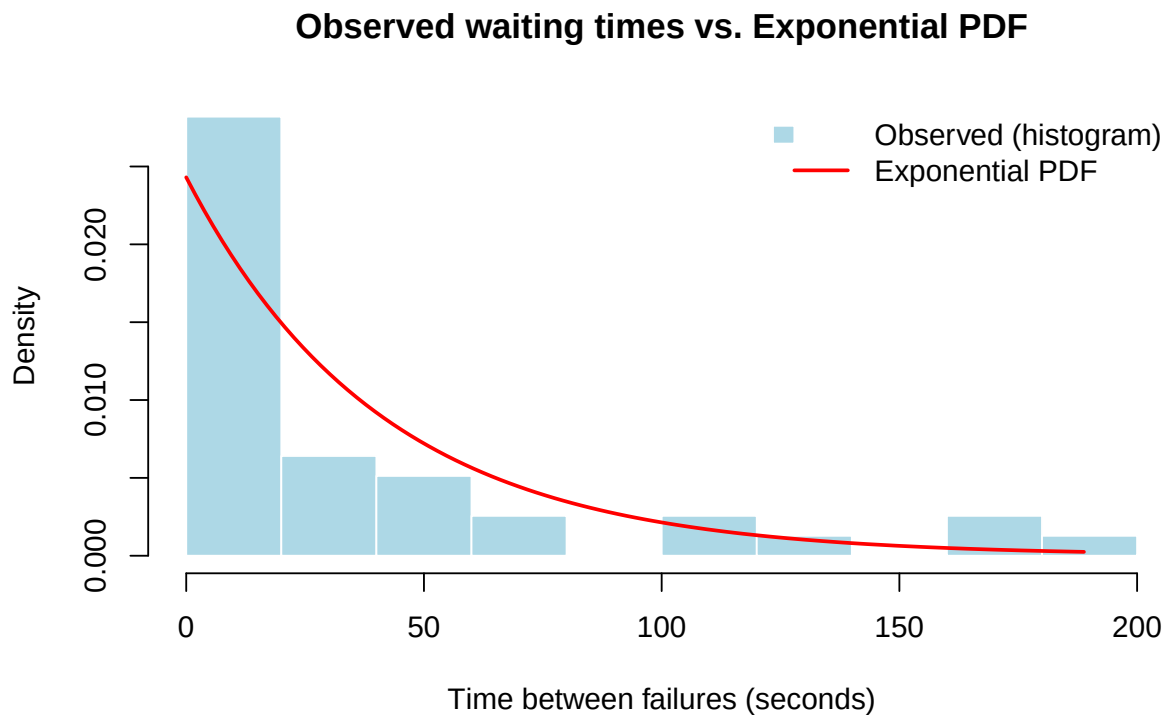
```
##   Time_sec P_within_t P_longer_than_t
## 1      10    0.2158    0.7842
## 2      30    0.5177    0.4823
## 3      60    0.7673    0.2327
## 4     120    0.9459    0.0541
```

```
# Cross-check using R's built-in exponential functions
round(pexp(times, rate = lambda), 4)
```

```
## [1] 0.2158 0.5177 0.7673 0.9459
```

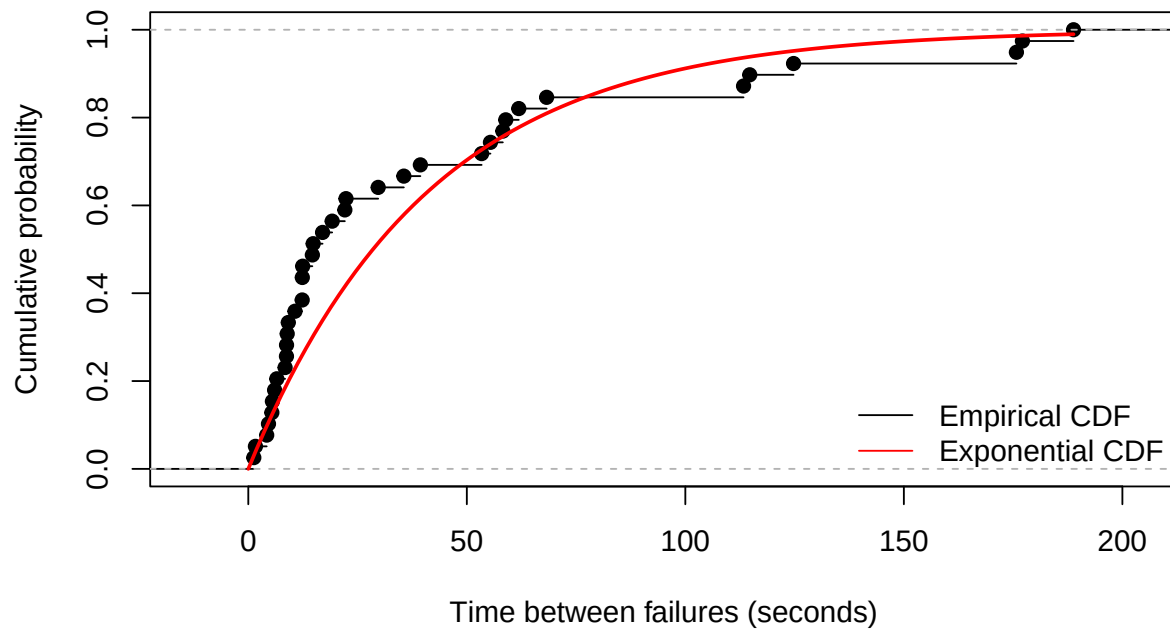
3. Visualize and Check the Fit

```
# Histogram of observed waiting times vs. fitted exponential PDF
hist(x, breaks = 10, freq = FALSE, col = "lightblue", border = "white",
     xlab = "Time between failures (seconds)",
     main = "Observed waiting times vs. Exponential PDF")
curve(dexp(x, rate = lambda), from = 0, to = max(x), add = TRUE,
      col = "red", lwd = 2)
legend("topright", legend = c("Observed (histogram)", "Exponential PDF"),
      fill = c("lightblue", NA), border = c("white", NA),
      lty = c(NA, 1), col = c(NA, "red"), lwd = c(NA, 2), bty = "n")
```

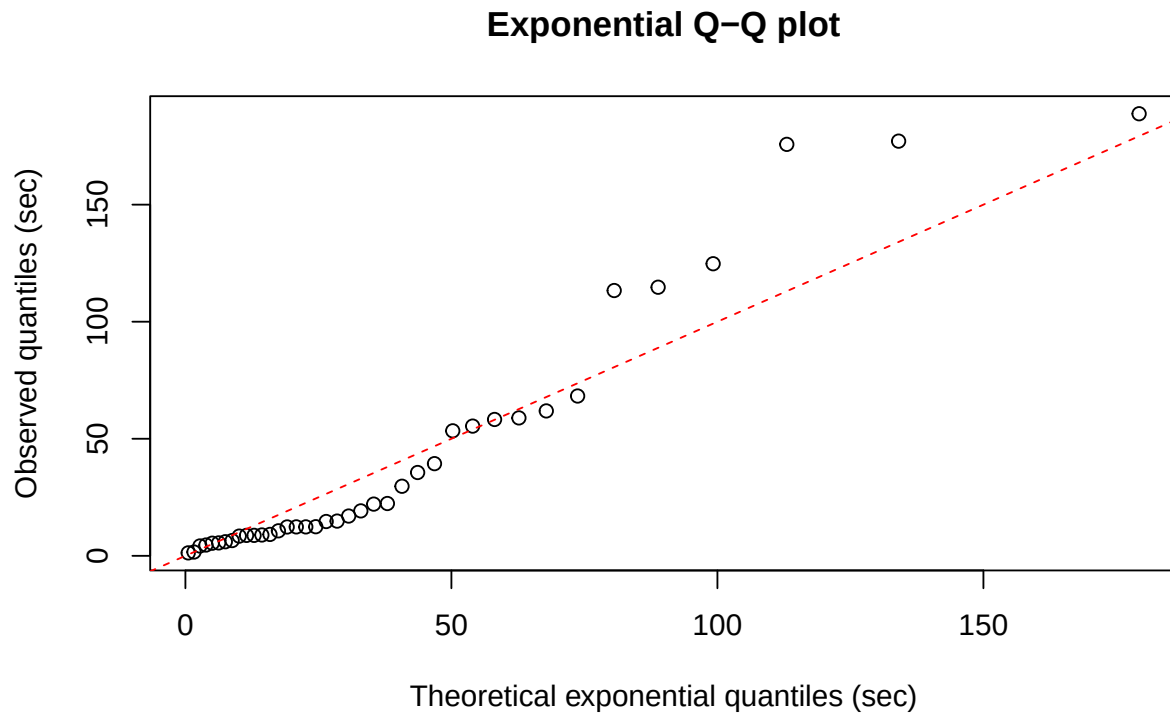


```
# Empirical CDF vs. fitted exponential CDF
plot(ecdf(x), main = "Empirical CDF vs. Exponential CDF",
     xlab = "Time between failures (seconds)", ylab = "Cumulative probability")
curve(pexp(x, rate = lambda), from = 0, to = max(x), add = TRUE,
      col = "red", lwd = 2)
legend("bottomright", legend = c("Empirical CDF", "Exponential CDF"),
      lty = 1, col = c("black", "red"), bty = "n")
```

Empirical CDF vs. Exponential CDF



```
# Q-Q plot against the exponential distribution
qqplot(qexp(ppoints(n), rate = lambda), x,
       xlab = "Theoretical exponential quantiles (sec)",
       ylab = "Observed quantiles (sec)",
       main = "Exponential Q-Q plot")
abline(0, 1, col = "red", lty = 2)
```



```
# Kolmogorov-Smirnov goodness-of-fit test
ks.test(x, "pexp", rate = lambda)
```

```
## Warning in ks.test.default(x, "pexp", rate = lambda): ties should not be
## present for the one-sample Kolmogorov-Smirnov test
```

```
##
## Asymptotic one-sample Kolmogorov-Smirnov test
##
## data: x
## D = 0.21003, p-value = 0.06407
## alternative hypothesis: two-sided
```

A p-value above 0.05 in the Kolmogorov-Smirnov test means there is no evidence against the exponential model, which supports using it for the waiting times between turnstile failures.

4. Report Findings

Chosen scenario. Turnstile failures at the security gates of the FEUture Center (FC) Building, defined as an unresponsive I.D. scanner or a face recognition failure. The random variable is the waiting time (in seconds) between one failure and the next.

Data collection process. The time gap between consecutive turnstile failures was recorded during peak student entry hours at around 12:00 noon, giving **39 observations**. Cumulative times were recomputed from the inter-arrival times and checked against the recorded values, then converted to hh:mm:ss timestamps.

Computed parameters.

##	Parameter	Value
## 1	Sample size (n)	39.0000
## 2	Rate (per second)	0.0243
## 3	Rate (per minute)	1.4580
## 4	Mean waiting time $E[X] = 1/$ (seconds)	41.1500
## 5	Lag-1 autocorrelation	0.2260

Graphs. See the cumulative failures plot, ACF plot, histogram with the fitted exponential PDF, empirical vs. exponential CDF, and Q-Q plot above.

Interpretation and real-world implications. See Section 5.

5. Interpret the Results

Determine the likelihood of turnstile failure in the FC Building

Using the Cumulative Distribution Function (CDF) for exponential waiting times, $P(X \leq t) = 1 - e^{-\lambda t}$, the likelihood that the next turnstile failure will occur within a given timeframe is:

- Within 10 seconds: **21.58%** chance
- Within 30 seconds: **51.77%** chance
- Within 60 seconds (1 minute): **76.73%** chance
- Within 120 seconds (2 minutes): **94.59%** chance

Implications

Because there is over a 76.7% chance that a failure will happen within a minute, and the average gap is only 41.15 seconds, system malfunctions occur frequently during peak student entry hours. This rapid succession of errors can quickly cause long queues and foot-traffic bottlenecks at the security gates of FEUTURE Center Building. As mitigation to this, a manual ID scanner is placed in the security gates. This also demands guards to be stationed, most especially in peak hours of student entry in FC Building. Furthermore, the maintenance of the turnstile systems should be done constantly to address these failures.